

Booking Experiment Analysis Script

Authors: Calvin Pan Created: June 8th 2026 Last modified: June 30th 2026

NOTE: Before using this script, please run your data through 00a-00c.py.

- 00a cleans all of the json files created by the **extension**, and outputs a CSV with all the events by all experimental subjects concatenated together.
- 00b joins this data from the extension with the choice set. (you have the option of taking the version of the choice set currently in the /config folder, or using a file in /inputs). It also gets rid of events that should not exist.
 - The script deletes hotels that are not in the choice set that pop up and get recorded when the extension refreshes Booking.com repeatedly (users never see those hotels).
 - The script also flags if users somehow manage to access features of Booking.com that they shouldn't have access to, like making their own searches.
- 00c joins the extension data with data from **oTree**. It creates a version of the data that is ready to analyze, with each observation being **subject x city x weekend x listing**. Each subject should have 3 cities x 4 weekends x 9 listings = 108 observations.
- The data is structured in this format so that you can use **groupby** to analyze at the level that you would like.

```
# Import cleaned data outputted by 00a-00c.py (data cleaning pipeline in Python)

# First, find filepaths
library(fs)

`%|||%` <- function(x, y) {
  if (is.null(x)) y else x
}

args <- commandArgs(trailingOnly = TRUE)

script_path <- if (requireNamespace("knitr", quietly = TRUE) &&
  !is.null(knitr::current_input())) {
  # Knitting: sys.frames()/rstudioapi don't reliably see the Rmd's own path
  # inside a knitr session, so ask knitr directly for the file it's rendering.
  knitr::current_input(dir = TRUE)
} else if (requireNamespace("rstudioapi", quietly = TRUE) &&
  rstudioapi::isAvailable()) {
  rstudioapi::getActiveDocumentContext()$path
} else {
  normalizePath(sys.frames()[[1]]$ofile %||% ".")
}

script_dir <- path_dir(path_dir(script_path)) # Define script directory

# Find inputs folder
INPUT_DIR <- path(script_dir, "inputs")

# Find output dir, output file
OUTPUT_DIR <- path(script_dir, "outputs")
CLEANED_CSV <- path(OUTPUT_DIR, "analysis_dataset.csv")
```

```

# Load packages used throughout the script
library(readr)
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(fixest) # fixed-effects OLS / logit / probit, clustered SE

## Warning: package 'fixest' was built under R version 4.4.3

library(ggplot2) # descriptives plots

# Load the cleaned analysis dataset (one row per subject x city x weekend x listing)
df <- read_csv(CLEANED_CSV, show_col_types = FALSE)

# Make sure ID variables used as fixed effects are factors
df <- df %>%
  mutate(
    participant_code = factor(participant_code),
    city = factor(city)
  )

glimpse(df)

## Rows: 6,650
## Columns: 34
## $ participant_code      <fct> 1dugzi2c, 1dugzi2c, 1dugzi2c, 1dugz~
## $ city                  <fct> la-ciotat, la-ciotat, la-ciotat, la~
## $ weekend_number_global <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2,~
## $ property_slug         <chr> "appartement-cocooning-avec-petite~
## $ clutter_treatment     <chr> "N", "N", "N", "N", "N", "N", "N", ~
## $ clutter_high          <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ cue_recognition        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
## $ nasa_tlx_mental        <dbl> 64, 64, 64, 64, 64, 64, 64, 64, 64,~
## $ nasa_tlx_physical      <dbl> 32, 32, 32, 32, 32, 32, 32, 32, 32,~
## $ nasa_tlx_temporal      <dbl> 82, 82, 82, 82, 82, 82, 82, 82, 82,~
## $ nasa_tlx_performance  <dbl> 76, 76, 76, 76, 76, 76, 76, 76, 76,~
## $ nasa_tlx_effort        <dbl> 65, 65, 65, 65, 65, 65, 65, 65, 65,~
## $ nasa_tlx_frustration  <dbl> 26, 26, 26, 26, 26, 26, 26, 26, 26,~
## $ visual_complexity      <dbl> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,~
## $ whole_experiment_time_seconds <dbl> 4576, 4576, 4576, 4576, 4576, 4576,~
## $ choice_task_time_seconds <dbl> 2814.999, 2814.999, 2814.999, 2814.~
## $ preference_consistency <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ cued_choice_preference_consistency <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA,~
## $ n_hotels_in_choice_set <dbl> 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9,~
## $ property_page_visits_number <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3,~
## $ decision_time_seconds <dbl> 120.576, 120.576, 120.576, 120.576,~

```

```
## $ loading_time_seconds      <dbl> 76.757, 76.757, 76.757, 76.757, 76.~
## $ cued_weekend             <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ listing_chosen           <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ listing_clicked          <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ listing_n_clicks         <dbl> 0, 0, 0, 0, 0, 0, 0, 2, 0, 1, 0, 0, ~
## $ time_on_listing_page_seconds <dbl> NA, NA, NA, NA, NA, NA, NA, 0.000, ~
## $ cluster                  <dbl> 3, 2, 2, 3, 2, 1, 1, 1, 3, 1, 1, 3, ~
## $ cued_listing             <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ checkin                  <date> 2026-10-16, 2026-10-16, 2026-10-16~
## $ choice_set               <chr> "choice_set_3", "choice_set_3", "ch~
## $ would_be_cued            <lgl> FALSE, FALSE, FALSE, TRUE, TRUE, TR~
## $ is_substitute_listing     <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, ~
## $ replaced_property_slug    <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
```

Sample exclusions

Per the pre-registration’s data inclusion and exclusion criteria, two rules apply to this dataset: (1) “Technical issues/link failures” — subjects who hit a technical issue that prevented them from completing all 12 choices (3 cities x 4 weekends) are fully excluded from the analysis (e.g. a **reserve** event that never got tracked for some weekend, or a session abandoned partway through)

- (2) “Reduced choice sets” — if a city x weekend choice set ends up with fewer than 9 listings, only the four weekends for that subject x city are dropped, while the subject’s other cities and post-experiment questionnaire variables remain in the analysis. The code below currently implements exclusion (1) only.

```
n_choices_per_subject <- df %>%
  group_by(participant_code) %>%
  summarise(
    n_choices = n_distinct(weekend_number_global[listing_chosen]),
    .groups = "drop"
  )

excluded_subjects <- n_choices_per_subject %>%
  filter(n_choices < 12) %>%
  pull(participant_code)

if (length(excluded_subjects) > 0) {
  cat("Excluding", length(excluded_subjects), "subject(s) with fewer than 12 choices made:\n")
  print(n_choices_per_subject %>% filter(participant_code %in% excluded_subjects))
} else {
  cat("No subjects excluded - every subject made all 12 choices.\n")
}
```

```
## Excluding 2 subject(s) with fewer than 12 choices made:
```

```
## # A tibble: 2 x 2
##   participant_code n_choices
##   <fct>          <int>
## 1 73c1d75p        10
## 2 bzmsdbca        9
```

Keep a pre-exclusion copy for the descriptives below - loading time is a technical/logistics measure independent of whether the subject completed all 12 choices, so it's reported on the full collected sample, not just the subjects that survive the exclusion.

```
df_all <- df
```

```

df <- df %>%
  filter(!participant_code %in% excluded_subjects) %>%
  droplevels()

# Reduced choice sets: if a city x weekend choice set ends up with fewer than 9
# listings, drop all 4 weekends for that subject x city pair (the subject's
# other cities and questionnaire variables are kept). Reuses 00c's own
# n_hotels_in_choice_set (nunique of property_slug per weekend) rather than
# recomputing a row count here, so this stays correct even if a future upstream
# change ever produces duplicate listing rows within a weekend.
listings_per_weekend <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(n_listings = first(n_hotels_in_choice_set), .groups = "drop")

reduced_choice_sets <- listings_per_weekend %>%
  filter(n_listings < 9) %>%
  distinct(participant_code, city)

if (nrow(reduced_choice_sets) > 0) {
  cat("Dropping", nrow(reduced_choice_sets),
      "subject x city pair(s) with a reduced (<9 listings) choice set in at least one weekend:\n")
  print(reduced_choice_sets)
} else {
  cat("No reduced choice sets (<9 listings) found - no subject x city pairs dropped.\n")
}

## Dropping 1 subject x city pair(s) with a reduced (<9 listings) choice set in at least one weekend:
## # A tibble: 1 x 2
##   participant_code city
##   <fct>           <fct>
## 1 68tpl2z9        la-ciotat

df <- df %>%
  anti_join(reduced_choice_sets, by = c("participant_code", "city")) %>%
  droplevels()

```

Descriptives — choice completeness & loading time

Two data-quality checks on the full collected sample (before the exclusion above): the share of subjects who did not complete all 12 choices, and how long Booking took to load the 9-listing choice set each weekend (the `loading_time_seconds` measure built in 00c from the extension's `preload` event).

```

# --- Share of subjects with fewer than 12 choices made ---
n_subjects_total    <- nrow(n_choices_per_subject)
n_subjects_excluded <- sum(n_choices_per_subject$n_choices < 12)
pct_excluded        <- 100 * n_subjects_excluded / n_subjects_total

cat(n_subjects_excluded, "of", n_subjects_total, "subjects (",
    round(pct_excluded, 1), "%) made fewer than 12 choices.\n\n")

## 2 of 62 subjects ( 3.2 %) made fewer than 12 choices.

# --- Loading time: weekend-level measure, so collapse the listing-level
# df_all (9 rows/weekend) down to one row per (participant_code,
# weekend_number_global) before summarising/plotting - otherwise every

```

```

# weekend's loading time would be counted 9 times over. ---
loading_by_weekend <- df_all %>%
  group_by(participant_code, weekend_number_global) %>%
  summarise(loading_time_seconds = first(loading_time_seconds), .groups = "drop")

n_missing_loading <- sum(is.na(loading_by_weekend$loading_time_seconds))

cat("Loading time (seconds), across", nrow(loading_by_weekend), "subject x weekend observations",
    if (n_missing_loading > 0) paste0(" (", n_missing_loading, " missing - no preload event tracked):\n")

## Loading time (seconds), across 739 subject x weekend observations (3 missing - no preload event tra
cat("  Mean:  ", round(mean(loading_by_weekend$loading_time_seconds, na.rm = TRUE), 2), "s\n")

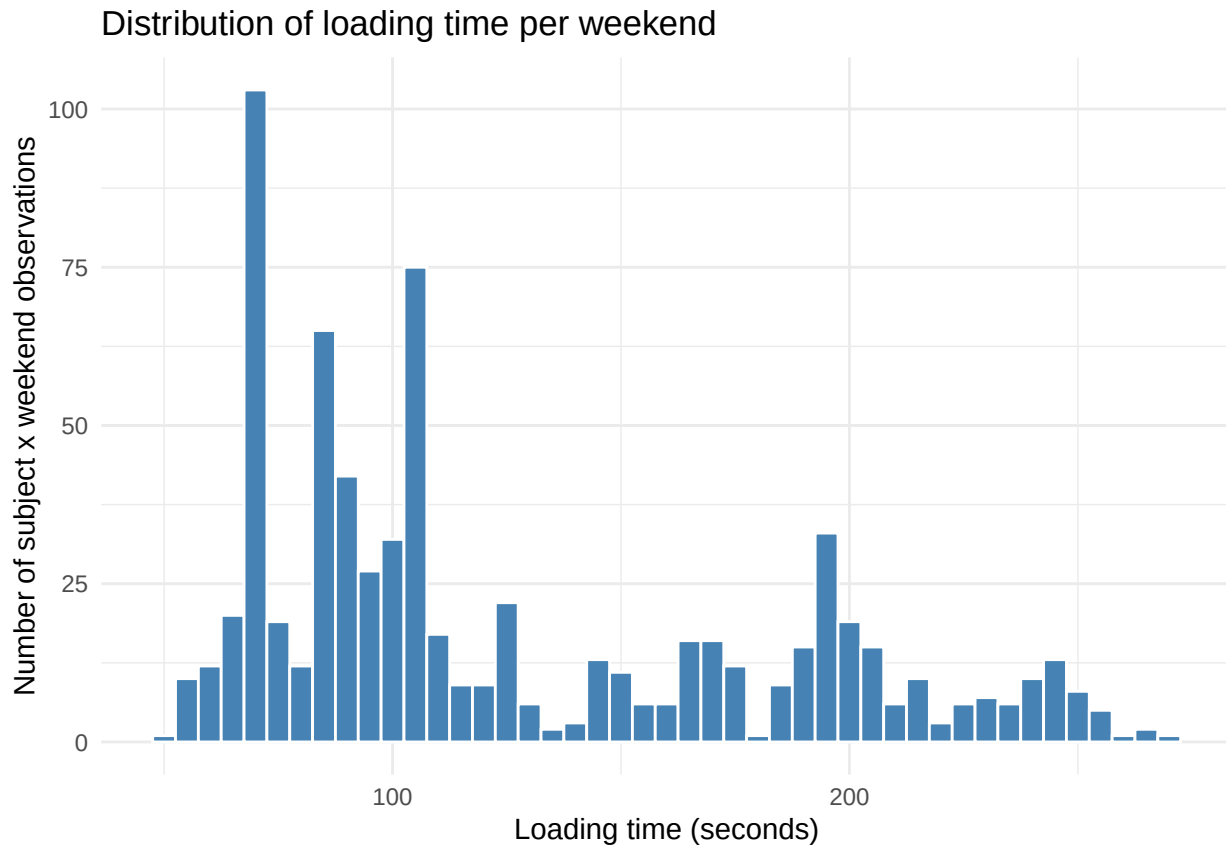
##   Mean:   125.77 s
cat("  Median:", round(median(loading_by_weekend$loading_time_seconds, na.rm = TRUE), 2), "s\n")

##   Median: 104.04 s
cat("  SD:    ", round(sd(loading_by_weekend$loading_time_seconds, na.rm = TRUE), 2), "s\n")

##   SD:     56.47 s

ggplot(loading_by_weekend, aes(x = loading_time_seconds)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white", na.rm = TRUE) +
  labs(
    title = "Distribution of loading time per weekend",
    x = "Loading time (seconds)",
    y = "Number of subject x weekend observations"
  ) +
  theme_minimal()

```



Testing H1.1 — Effect of the cue on choice probability

We test H1.1 using a fixed-effects logistic regression. The dependent variable is “Cue-eligible listing chosen”, and the key predictor is “Endorsement Cue Visibility” (Treatment 1). The model includes subject and city fixed effects, which control for stable differences across people and across cities that are unrelated to the treatment itself. Our unit of analysis is the choice set: each subject contributes one observation per city per weekend, for a total of 12 observations per subject. H1.1 predicts that the cue increases the probability of choosing cue-eligible listings, so we use a one-sided test on the coefficient of interest ($H_0 : \beta \leq 0$, $H_1 : \beta > 0$), with a Type I error rate of $\alpha = 0.10$ on the positive tail.

```
# Collapse listing-level rows (9 per weekend) to one row per subject x city x weekend,
# since H1.1's DV is defined at the weekend level, not the listing level.
h11_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    # First, construction of the DV. In every single choice set, there are 3 listings out of the 9
    # (1 per cluster) that are ones that in the real-world Booking.com environment have a thumb:
    # i.e. they would have a cue if the subject is assigned to see cues that weekend. We are trying
    # to compare if the probability that a subject chooses a "would_be_cued" listing is higher
    # when the cue is visible vs. not. As such, chosen_cued_listing, the DV, is TRUE if the subject
    # chose a would_be_cued listing.
    chose_cued_listing = any(listing_chosen & would_be_cued, na.rm = TRUE),
    # treatment 1: TRUE if the cue graphic was actually shown this weekend (cell_thumb == "P")
    cue_visible = first(cued_weekend),
    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code), city = factor(city))
```

```

# Fixed-effects logit: feglm() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only.
m_h11 <- feglm(
  chose_cued_listing ~ cue_visible | participant_code + city,
  data = h11_df,
  family = "logit"
)

summary(m_h11)

## GLM estimation, family = binomial(link = "logit")
## Dep. Var.: chose_cued_listing
## Observations: 716
## Fixed-effects: participant_code: 60, city: 4
## Standard-errors: IID
##               Estimate Std. Error  z value Pr(>|z|)
## cue_visibleTRUE 0.158081   0.170099 0.929346  0.35271
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood: -438.3   Adj. Pseudo R2: -0.063554
##               BIC: 1,297.3   Squared Cor.: 0.089002

# summary() above gives a two-sided p-value (H0: beta = 0); H1.1 predicts a
# specific direction (H0: beta <= 0, H1: beta > 0), so compute the one-sided
# p-value manually from the z-statistic.
coef_h11 <- coef(m_h11)["cue_visibleTRUE"]
se_h11    <- se(m_h11)["cue_visibleTRUE"]
z_h11     <- coef_h11 / se_h11
p_onesided_h11 <- pnorm(z_h11, lower.tail = FALSE)

cat("z =", round(z_h11, 3), " | one-sided p-value (H1: beta > 0) =", round(p_onesided_h11, 4), "\n")

## z = 0.929 | one-sided p-value (H1: beta > 0) = 0.1764

```

Testing H1.2 — Effect of the cue on attention

We test H1.2.1 using a fixed-effects logistic regression on “Cue-eligible listing clicked” — whether at least one cue-eligible listing’s detail page is clicked on by the subject within a choice set (city x weekend) — and H1.2.2 using an OLS regression on “Time on the detail page of cue-eligible listings” — the log(seconds + 1) of the summed time spent across cue-eligible listings’ pages within a choice set, coded as zero if no time was spent on any cue-eligible detail page. The key predictor in both models is “Endorsement Cue Visibility” (Treatment 1). Both models include subject and city fixed effects, which control for stable differences across people and across cities that are unrelated to the treatment itself. Our unit of analysis is the choice set: each subject contributes one observation per city per weekend, for a total of 12 observations per subject. H1.2 predicts that the cue increases attention to cue-eligible listings, so we use one-sided tests on the coefficient of interest ($H_0 : 1 \leq 0$, $H_1 : 1 > 0$), with a Type I error rate of $\alpha = 0.10$ on the positive tail.

```

# Collapse listing-level rows (9 per weekend) to one row per subject x city x weekend:
# "Cue-eligible listing clicked" is defined as whether AT LEAST ONE cue-eligible
# listing is clicked within a choice set (city x weekend), built the same way
# H1.1's DV was.
h121_df <- df %>%

```

```

group_by(participant_code, city, weekend_number_global) %>%
summarise(
  cue_listing_clicked = any(listing_clicked & would_be_cued, na.rm = TRUE),
  cue_visible = first(cued_weekend),
  .groups = "drop"
) %>%
mutate(participant_code = factor(participant_code), city = factor(city))

# Fixed-effects logit: feglm() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only.
m_h121 <- feglm(
  cue_listing_clicked ~ cue_visible | participant_code + city,
  data = h121_df,
  family = "logit"
)

```

H1.2.1 first: Binary outcome “whether cued listing is clicked on” with logit & fixed effects

NOTE: 1/0 fixed-effect (12 observations) removed because of only 0 or only 1 outcomes or singletons.

```
summary(m_h121)
```

```

## GLM estimation, family = binomial(link = "logit")
## Dep. Var.: cue_listing_clicked
## Observations: 704
## Fixed-effects: participant_code: 59, city: 4
## Standard-errors: IID
##               Estimate Std. Error z value Pr(>|z|)
## cue_visibleTRUE 0.265111  0.170854 1.55168  0.12074
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood: -434.3   Adj. Pseudo R2: -0.026615
##               BIC: 1,281.6   Squared Cor.: 0.131418

```

*# summary() above gives a two-sided p-value (H0: beta = 0); H1.2 predicts a
specific direction (H0: beta ≤ 0, H1: beta > 0), so compute the one-sided
p-value manually from the z-statistic.*

```

coef_h121 <- coef(m_h121)["cue_visibleTRUE"]
se_h121    <- se(m_h121)["cue_visibleTRUE"]
z_h121     <- coef_h121 / se_h121
p_onesided_h121 <- pnorm(z_h121, lower.tail = FALSE)

```

```
cat("z =", round(z_h121, 3), " | one-sided p-value (H1: beta > 0) =", round(p_onesided_h121, 4), "\n")
```

```
## z = 1.552 | one-sided p-value (H1: beta > 0) = 0.0604
```

*# Choice-set level: "Time on the detail page of cue-eligible listings" is the
SUM of seconds across the (up to 3) cue-eligible listings within a weekend
(0 if none were clicked/no time recorded), then log(seconds + 1)-transformed
per the pre-registration's Transformations section.*

```

h122_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    cue_time_seconds = sum(time_on_listing_page_seconds[would_be_cued], na.rm = TRUE),

```



```

    cue_visible = first(cued_weekend),
    .groups = "drop"
  ) %>%
  mutate(
    participant_code = factor(participant_code),
    city = factor(city),
    log_cue_time_seconds = log(cue_time_seconds + 1)
  )

# Fixed-effects OLS: feols() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only.
m_h122 <- feols(
  log_cue_time_seconds ~ cue_visible | participant_code + city,
  data = h122_df
)

summary(m_h122)

```

H1.2.2 first: OLS “time on detail page of listing” with fixed effects

```

## OLS estimation, Dep. Var.: log_cue_time_seconds
## Observations: 716
## Fixed-effects: participant_code: 60, city: 4
## Standard-errors: IID
##               Estimate Std. Error t value Pr(>|t|)
## cue_visibleTRUE 0.000472   0.000339 1.39224  0.16433
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.004328      Adj. R2: 0.137778
##               Within R2: 0.002964

# summary() above gives a two-sided p-value (H0: beta = 0); H1.2 predicts a
# specific direction (H0: beta <= 0, H1: beta > 0), so compute the one-sided
# p-value manually from the z-statistic.
coef_h122 <- coef(m_h122)["cue_visibleTRUE"]
se_h122    <- se(m_h122)["cue_visibleTRUE"]
z_h122     <- coef_h122 / se_h122
p_onesided_h122 <- pnorm(z_h122, lower.tail = FALSE)

cat("z =", round(z_h122, 3), " | one-sided p-value (H1: beta > 0) =", round(p_onesided_h122, 4), "\n")

## z = 1.392 | one-sided p-value (H1: beta > 0) = 0.0819

```

Testing H2.1 — Clutter as moderator of the cue effect

We test H2.1 using the same models described above. For binary outcomes — Whether the cue-eligible listing is chosen and Whether the cue-eligible listing is clicked on — we use fixed-effects logistic regression. For the continuous outcome Time spent on the detail page of the cue-eligible listing, we use OLS with fixed effects. To test whether clutter moderates the effect of the cue, we augment each model with an interaction term between: • Treatment 1 (Endorsement Cue Visibility): whether the cue is displayed, • Treatment 2 (Visual Clutter): whether the interface is in the Cluttered condition. Because the clutter condition is between-subject and constant within subject, the subject fixed effect absorbs its main effect; we are therefore only able to identify the interaction, not the main effect of clutter. Since H2.1 does not specify the direction of the interaction effect, we use a two-sided test on the interaction coefficient ($H_0 : \beta_{12} = 0$, $H_1 : \beta_{12} \neq 0$),

with a Type I error rate of $\alpha = 0.10$ split across both tails.

```
# Collapse listing-level rows (9 per weekend) to one row per subject x city x weekend, since we're working with h211_df
h211_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    # Like before, DV, chosen_cued_listing, is TRUE if the subject chose a would_be_cued listing.
    chose_cued_listing = any(listing_chosen & would_be_cued, na.rm = TRUE),
    # treatment 1: TRUE if the cue graphic was actually shown this weekend (cell_thumb == "P")
    cue_visible = first(cued_weekend),
    # clutter_high is subject-level constant; first() avoids the same row-duplication
    # bug we hit in h22_df/h231_df (bare column references aren't aggregated by summarise()).
    clutter_high = first(clutter_high),
    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code), city = factor(city))

# Fixed-effects logit: feglm() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only. This time, interaction between cue_visible and clutter_high
m_h211 <- feglm(
  chose_cued_listing ~ cue_visible * clutter_high | participant_code + city,
  data = h211_df,
  family = "logit"
)
```

H2.1.1 DV: Whether cued listing is chosen, binary logit with fixed effects and interaction

```
## The variable 'clutter_highTRUE' has been removed because of collinearity (see
## $collin.var).
```

```
summary(m_h211)
```

```
## GLM estimation, family = binomial(link = "logit")
## Dep. Var.: chose_cued_listing
## Observations: 716
## Fixed-effects: participant_code: 60, city: 4
## Standard-errors: IID
##
##               Estimate Std. Error  z value Pr(>|z|)
## cue_visibleTRUE      0.025330   0.235885  0.107382  0.91449
## cue_visibleTRUE:clutter_highTRUE 0.276945   0.341037  0.812067  0.41675
## ... 1 variable was removed because of collinearity (clutter_highTRUE)
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood:  -437.9    Adj. Pseudo R2: -0.064907
##               BIC: 1,303.1    Squared Cor.:  0.090607
```

```
# Choice-set level, same DV construction as H1.2.1: "at least one cue-eligible
# listing clicked" per choice set (city x weekend), with the clutter interaction added.
h212_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    cue_listing_clicked = any(listing_clicked & would_be_cued, na.rm = TRUE),
```

```

    cue_visible = first(cued_weekend),
    clutter_high = first(clutter_high),
    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code), city = factor(city))

# Fixed-effects logit: feglm() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only. Interaction added too.
m_h212 <- feglm(
  cue_listing_clicked ~ cue_visible * clutter_high | participant_code + city,
  data = h212_df,
  family = "logit"
)

```

H2.1.2 DV: Whether cued listing is clicked on, binary logit with fixed effects and interaction

NOTE: 1/0 fixed-effect (12 observations) removed because of only 0 or only 1 outcomes or singletons.

The variable 'clutter_highTRUE' has been removed because of collinearity (see
\$collin.var).

```
summary(m_h212)
```

```

## GLM estimation, family = binomial(link = "logit")
## Dep. Var.: cue_listing_clicked
## Observations: 704
## Fixed-effects: participant_code: 59,  city: 4
## Standard-errors: IID
##
##              Estimate Std. Error  z value Pr(>|z|)
## cue_visibleTRUE      0.254438   0.236840  1.074305  0.28269
## cue_visibleTRUE:clutter_highTRUE 0.022284   0.342264  0.065108  0.94809
## ... 1 variable was removed because of collinearity (clutter_highTRUE)
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood:  -434.3   Adj. Pseudo R2: -0.028678
##              BIC: 1,288.2   Squared Cor.:  0.13143

```

*# Choice-set level, same DV construction as H1.2.2: log(seconds + 1) of the
summed cue-eligible listing time within a choice set, with the clutter
interaction added.*

```

h213_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    cue_time_seconds = sum(time_on_listing_page_seconds[would_be_cued], na.rm = TRUE),
    cue_visible = first(cued_weekend),
    clutter_high = first(clutter_high),
    .groups = "drop"
  ) %>%
  mutate(
    participant_code = factor(participant_code),
    city = factor(city),
    log_cue_time_seconds = log(cue_time_seconds + 1)
  )

```

```

# Fixed-effects OLS: feols() absorbs participant_code and city fixed effects
# (instead of estimating a dummy per subject/city), so cue_visible is identified
# off within-subject, within-city variation only.
m_h213 <- feols(
  log_cue_time_seconds ~ cue_visible * clutter_high | participant_code + city,
  data = h213_df
)

```

H2.1.3 DV: Time spent on detail page, OLS with fixed effects and interaction

```

## The variable 'clutter_highTRUE' has been removed because of collinearity (see
## $collin.var).

```

```
summary(m_h213)
```

```

## OLS estimation, Dep. Var.: log_cue_time_seconds
## Observations: 716
## Fixed-effects: participant_code: 60, city: 4
## Standard-errors: IID
##
##               Estimate Std. Error   t value Pr(>|t|)
## cue_visibleTRUE      0.000625   0.000471   1.327894  0.18468
## cue_visibleTRUE:clutter_highTRUE -0.000318   0.000679  -0.468834  0.63934
## ... 1 variable was removed because of collinearity (clutter_highTRUE)
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.004327      Adj. R2: 0.136745
##
##               Within R2: 0.003301

```

Testing H2.2 — Dilution mechanism

We test H2.2 using a logistic regression. The dependent variable is the binary indicator “Cue recognition”. The key predictor is “Visual Clutter” (Treatment 2). We have one observation per subject. H2.2 predicts that clutter reduces cue recognition, so we use a one-sided test ($H_0: \beta \geq 0$, $H_1: \beta < 0$), with a Type I error rate of $\alpha = 0.10$ on the negative tail.

```

# Collapse listing-level to one row per subject only, since we're working at the subject level.
# cue_recognition and clutter_high are subject-level constants (same value on every one of the
# subject's 108 rows), so first() picks that single value instead of keeping all repeats.
h22_df <- df %>%
  group_by(participant_code) %>%
  summarise(
    cue_recognition = first(cue_recognition),
    clutter_high = first(clutter_high),
    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code))

# Logit: glm(), no fixed effects (1 observation per subject - no panel structure to absorb).
m_h22 <- glm(
  cue_recognition ~ clutter_high,
  data = h22_df,
  family = binomial(link = "logit")
)

summary(m_h22)

```

```
##
## Call:
## glm(formula = cue_recognition ~ clutter_high, family = binomial(link = "logit"),
##      data = h22_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.45953    0.36873   1.246   0.213
## clutter_highTRUE 0.03294    0.53144   0.062   0.951
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 79.881  on 59  degrees of freedom
## Residual deviance: 79.877  on 58  degrees of freedom
## AIC: 83.877
##
## Number of Fisher Scoring iterations: 4

# H2.2 predicts clutter REDUCES cue recognition (H0: beta1 >= 0, H1: beta1 < 0),
# so the one-sided p-value is the area to the LEFT of the z-statistic.
coef_h22 <- coef(m_h22)["clutter_highTRUE"]
se_h22    <- summary(m_h22)$coefficients["clutter_highTRUE", "Std. Error"]
z_h22     <- coef_h22 / se_h22
p_onesided_h22 <- pnorm(z_h22, lower.tail = TRUE)

cat("z =", round(z_h22, 3), " | one-sided p-value (H1: beta < 0) =", round(p_onesided_h22, 4), "\n")

## z = 0.062 | one-sided p-value (H1: beta < 0) = 0.5247
```

Testing H2.3 — Decision mode mechanism

We test H2.3 using a set of OLS regressions. The dependent variables are (i) decision time and (ii) number of listing page visits. The key predictor is “Visual Clutter” (Treatment 2). All specifications include city-level fixed effects to control for location-specific factors. We additionally test the effect of clutter on the NASA-TLX items on effort, performance, and mental demand; since the NASA-TLX is collected once per participant at the end of the experiment, this yields one observation per subject, so we estimate this model using OLS without fixed effects and without clustering. H2.3 predicts that clutter induces greater reliance on heuristics. We interpret lower decision time and fewer listing page visits as indicative of heuristic use. We interpret lower effort with no lower mental demand in the Cluttered condition as indicative of heuristic use. We interpret higher expected performance for no lower effort in the Cluttered condition as rejecting the use of heuristics. Other configurations of the NASA-TLX items are interpreted as ambiguous. We use one-sided tests throughout: we expect lower decision time and fewer page visits under high clutter ($H_0: 1 \leq 0$, $H_1: 1 < 0$), and higher perceived cognitive load ($H_0: 1 \leq 0$, $H_1: 1 > 0$), each with a Type I error rate of $\alpha = 0.10$ on the respective tail.

```
# Weekend level: decision_time_seconds and clutter_high are both constant across
# the 9 listing-rows of a weekend, so first() picks that single value per group.
# log(seconds + 1)-transformed per the pre-registration's Transformations section.
h231_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    decision_time_seconds = first(decision_time_seconds),
    clutter_high = first(clutter_high),
    .groups = "drop"
```

```

) %>%
mutate(
  participant_code = factor(participant_code),
  city = factor(city),
  log_decision_time_seconds = log(decision_time_seconds + 1)
)

# NOTE: clutter_high is a BETWEEN-subject treatment - it never varies across a
# given subject's weekends/cities, so it is perfectly collinear with the
# participant_code fixed effect (each subject dummy already fully encodes their
# own clutter_high). Including participant_code here leaves zero residual
# variation to estimate beta, so we drop it and keep only the city fixed effect
# (this matches the pre-registration, which only specifies city-level fixed
# effects for H2.3).

m_h231 <- feols(
  log_decision_time_seconds ~ clutter_high | city,
  data = h231_df
)

```

H2.3.1: DV: Decision time, IV: Clutter, OLS with fixed effects

NOTE: 3 observations removed because of NA values (LHS: 3).

```
summary(m_h231)
```

```

## OLS estimation, Dep. Var.: log_decision_time_seconds
## Observations: 713
## Fixed-effects: city: 4
## Standard-errors: IID
##
##           Estimate Std. Error t value Pr(>|t|)
## clutter_highTRUE 0.096783   0.055287 1.75055 0.080457 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.733972      Adj. R2: 0.084764
##
##           Within R2: 0.00431

```

*# H2.3 predicts LOWER decision time under high clutter (H0: $\beta_1 \geq 0$, H1: $\beta_1 < 0$),
so the one-sided p-value is the area to the LEFT of the t-statistic.*

```

coef_h231 <- coef(m_h231)["clutter_highTRUE"]
se_h231    <- se(m_h231)["clutter_highTRUE"]
t_h231     <- coef_h231 / se_h231
p_onesided_h231 <- pnorm(t_h231, lower.tail = TRUE)

```

```
cat("t =", round(t_h231, 3), " | one-sided p-value (H1:  $\beta < 0$ ) =", round(p_onesided_h231, 4), "\n")
```

```
## t = 1.751 | one-sided p-value (H1:  $\beta < 0$ ) = 0.96
```

*# Weekend level: property_page_visits_number and clutter_high are both constant across
the 9 listing-rows of a weekend, so first() picks that single value per group.*

```

h232_df <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(
    property_page_visits_number = first(property_page_visits_number),
    clutter_high = first(clutter_high),

```

```

    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code), city = factor(city))

# NOTE: clutter_high is a BETWEEN-subject treatment - it never varies across a
# given subject's weekends/cities, so it is perfectly collinear with the
# participant_code fixed effect (each subject dummy already fully encodes their
# own clutter_high). Including participant_code here leaves zero residual
# variation to estimate beta, so we drop it and keep only the city fixed effect
# (this matches the pre-registration, which only specifies city-level fixed
# effects for H2.3).
m_h232 <- feols(
  property_page_visits_number ~ clutter_high | city,
  data = h232_df
)

summary(m_h232)

```

H2.3.2: DV: Number of property page visits, IV: Clutter, OLS with fixed effect

```

## OLS estimation, Dep. Var.: property_page_visits_number
## Observations: 716
## Fixed-effects: city: 4
## Standard-errors: IID
##
##              Estimate Std. Error t value   Pr(>|t|)
## clutter_highTRUE 0.420939   0.117873  3.57113 0.00037939 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 1.56823      Adj. R2: 0.053452
##
##              Within R2: 0.017621

# H2.3 predicts LOWER decision time under high clutter (H0: beta1 >= 0, H1: beta1 < 0),
# so the one-sided p-value is the area to the LEFT of the t-statistic.
coef_h232 <- coef(m_h232)["clutter_highTRUE"]
se_h232    <- se(m_h232)["clutter_highTRUE"]
t_h232     <- coef_h232 / se_h232
p_onesided_h232 <- pnorm(t_h232, lower.tail = TRUE)

cat("t =", round(t_h232, 3), " | one-sided p-value (H1: beta < 0) =", round(p_onesided_h232, 4), "\n")

## t = 3.571 | one-sided p-value (H1: beta < 0) = 0.9998

```

```

# NASA-TLX is collected once per participant -> collapse to 1 row/subject.
nasa_vars <- c("nasa_tlx_mental", "nasa_tlx_physical", "nasa_tlx_temporal",
              "nasa_tlx_performance", "nasa_tlx_effort", "nasa_tlx_frustration")

h233_df <- df %>%
  group_by(participant_code) %>%
  summarise(
    clutter_high = first(clutter_high),
    across(all_of(nasa_vars), first),
    .groups = "drop"
  )

```

```

# Loop over each NASA-TLX dimension: OLS, no FE (1 row/subject = no panel
# structure to absorb) and no clustering (same reason - see H2.2).
m_h233_list <- setNames(vector("list", length(nasa_vars)), nasa_vars)
h233_results <- data.frame()

for (v in nasa_vars) {
  fml <- as.formula(paste(v, "~ clutter_high"))
  m <- feols(fml, data = h233_df)
  m_h233_list[[v]] <- m

  # H2.3 predicts HIGHER cognitive load under high clutter (H0: beta1 <= 0,
  # H1: beta1 > 0), so the one-sided p-value is the area to the RIGHT of t.
  b <- coef(m)["clutter_highTRUE"]
  std_err <- se(m)["clutter_highTRUE"]
  t_stat <- b / std_err
  p_onesided <- pnorm(t_stat, lower.tail = FALSE)

  h233_results <- rbind(h233_results, data.frame(
    dv = v, estimate = b, std_error = std_err, t_stat = t_stat, p_onesided = p_onesided
  ))
}

# esttab()-style side-by-side regression table - fixest's equivalent.
etable(m_h233_list)

```

H2.3.3: NASA-TLX dimensions, OLS without fixed effects

```

##               nasa_tlx_mental nasa_tlx_physical nasa_tlx_temporal
## Dependent Var.: nasa_tlx_mental nasa_tlx_physical nasa_tlx_temporal
##
## Constant      45.16*** (5.392)  27.55*** (4.297)  37.74*** (5.123)
## clutter_highTRUE  4.184 (7.755)   -1.583 (6.181)   12.71. (7.369)
## -----
## S.E. type      IID              IID              IID
## Observations    60              60              60
## R2              0.00499          0.00113          0.04877
## Adj. R2         -0.01216         -0.01609          0.03236
##
##               nasa_tlx_performance nasa_tlx_effort nasa_tlx_frustration
## Dependent Var.: nasa_tlx_performance nasa_tlx_effort nasa_tlx_frustration
##
## Constant      80.52*** (3.246)  44.13*** (5.117)   26.71*** (4.243)
## clutter_highTRUE -3.516 (4.669)  -1.922 (7.361)    1.083 (6.103)
## -----
## S.E. type      IID              IID              IID
## Observations    60              60              60
## R2              0.00968          0.00117          0.00054
## Adj. R2         -0.00739         -0.01605         -0.01669
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Compact one-sided-test summary across all 6 dimensions.
print(h233_results, row.names = FALSE)

```

```

##               dv estimate std_error      t_stat p_onesided

```



```
##      nasa_tlx_mental  4.183537  7.755227  0.5394474  0.29478908
##      nasa_tlx_physical -1.582870  6.180761 -0.2560963  0.60106175
##      nasa_tlx_temporal 12.706340  7.368744  1.7243564  0.04232177
##      nasa_tlx_performance -3.516129  4.668655 -0.7531354  0.77431572
##      nasa_tlx_effort -1.922136  7.360551 -0.2611402  0.60300780
##      nasa_tlx_frustration 1.083426  6.103245  0.1775164  0.42955140
```

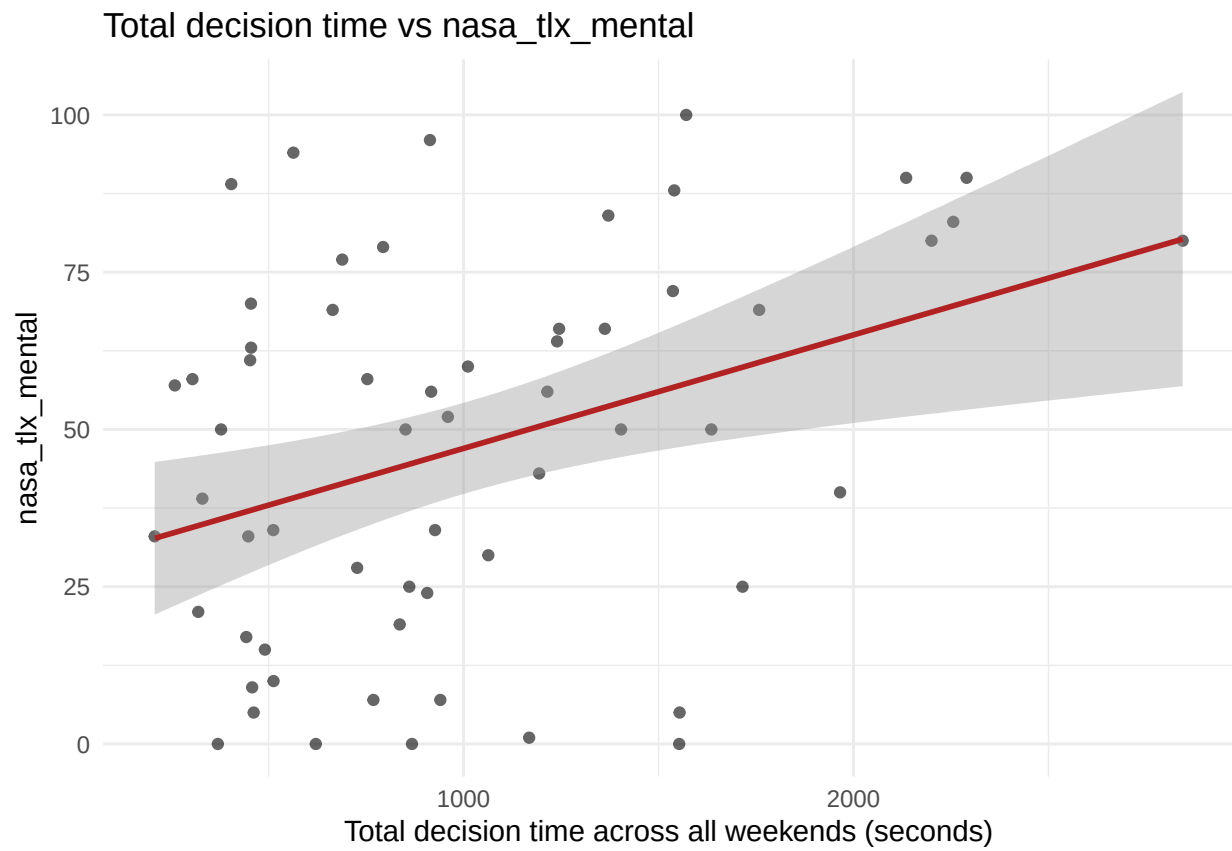
```
# Exploratory/descriptive only - not a pre-registered analysis. Subject-level
# total decision time: raw (unlogged) decision_time_seconds summed across all
# 12 weekends per subject (weekends with no tracked preload event, and thus no
# decision_time_seconds, contribute 0 via na.rm = TRUE rather than dropping the
# subject entirely).
decision_time_by_subject <- df %>%
  group_by(participant_code, city, weekend_number_global) %>%
  summarise(decision_time_seconds = first(decision_time_seconds), .groups = "drop") %>%
  group_by(participant_code) %>%
  summarise(total_decision_time_seconds = sum(decision_time_seconds, na.rm = TRUE), .groups = "drop")

audit_df <- h233_df %>%
  select(participant_code, all_of(nasa_vars)) %>%
  left_join(decision_time_by_subject, by = "participant_code")

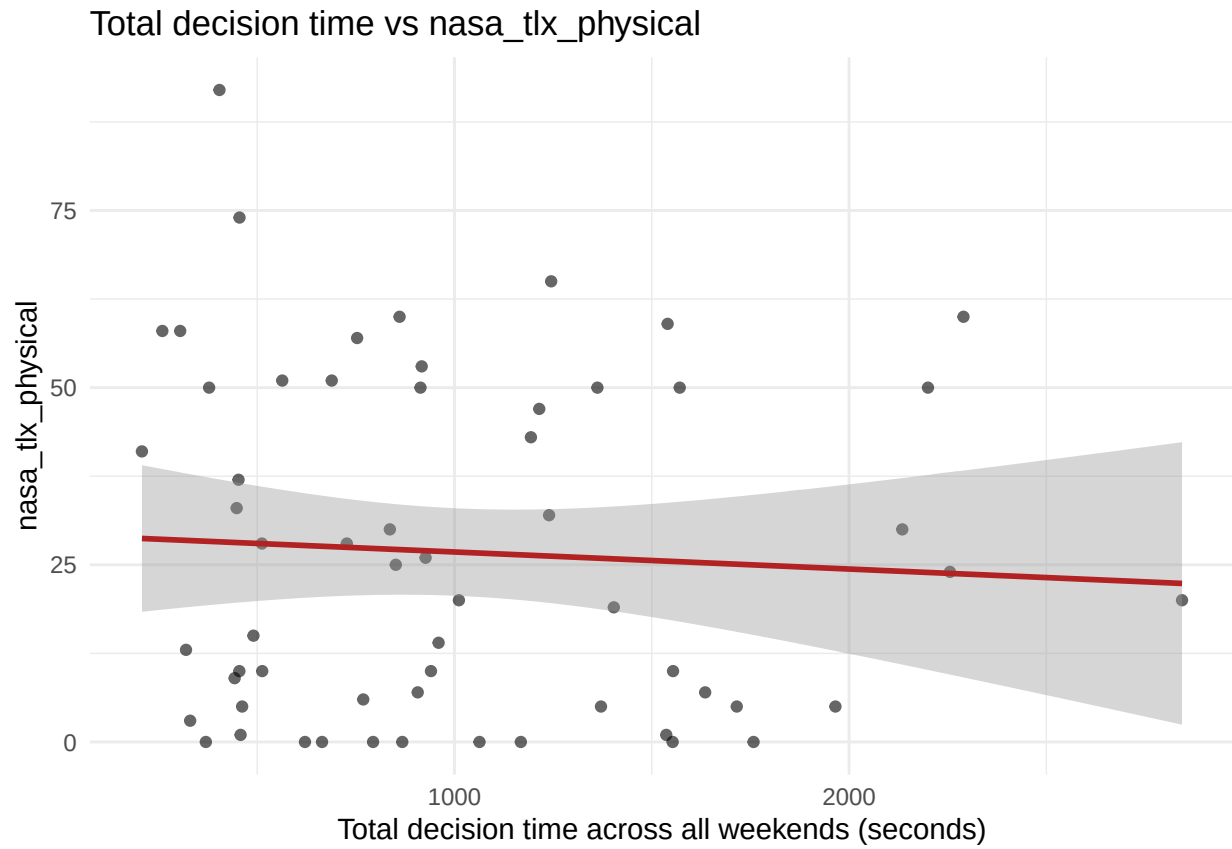
for (v in nasa_vars) {
  p <- ggplot(audit_df, aes(x = total_decision_time_seconds, y = .data[[v]])) +
    geom_point(alpha = 0.6) +
    geom_smooth(method = "lm", se = TRUE, color = "firebrick") +
    labs(
      title = paste("Total decision time vs", v),
      x = "Total decision time across all weekends (seconds)",
      y = v
    ) +
    theme_minimal()
  print(p)
}
```

Audit: decision_time_seconds (sum) vs NASA-TLX dimensions

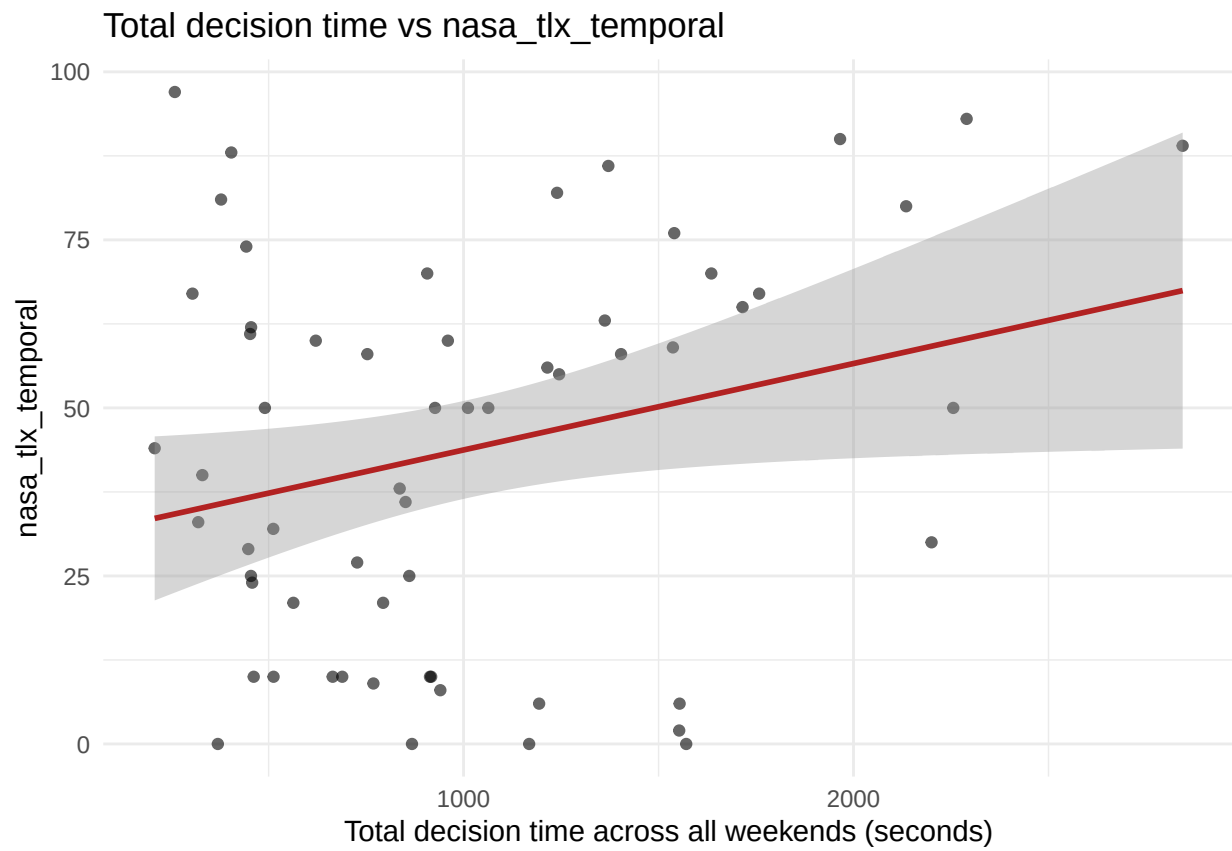
```
## `geom_smooth()` using formula = 'y ~ x'
```



```
## `geom_smooth()` using formula = 'y ~ x'
```

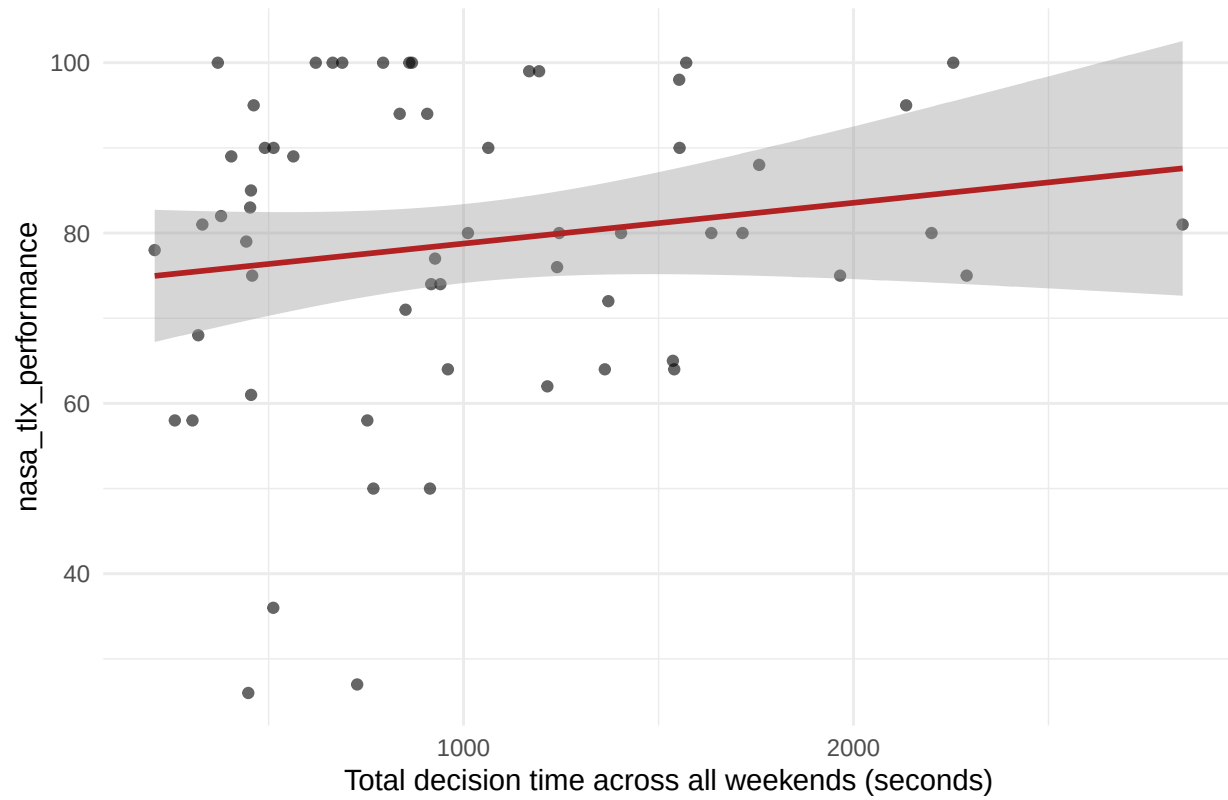


```
## `geom_smooth()` using formula = 'y ~ x'
```



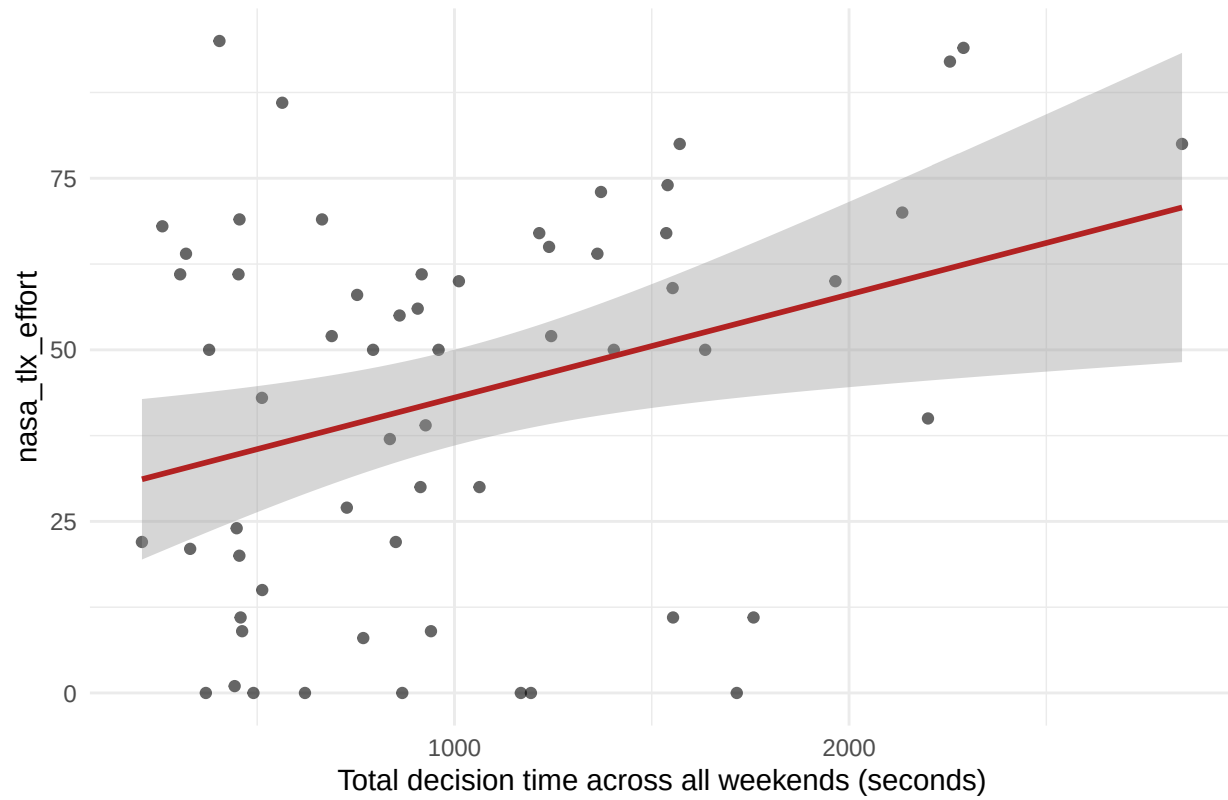
```
## `geom_smooth()` using formula = 'y ~ x'
```

Total decision time vs nasa_tlx_performance

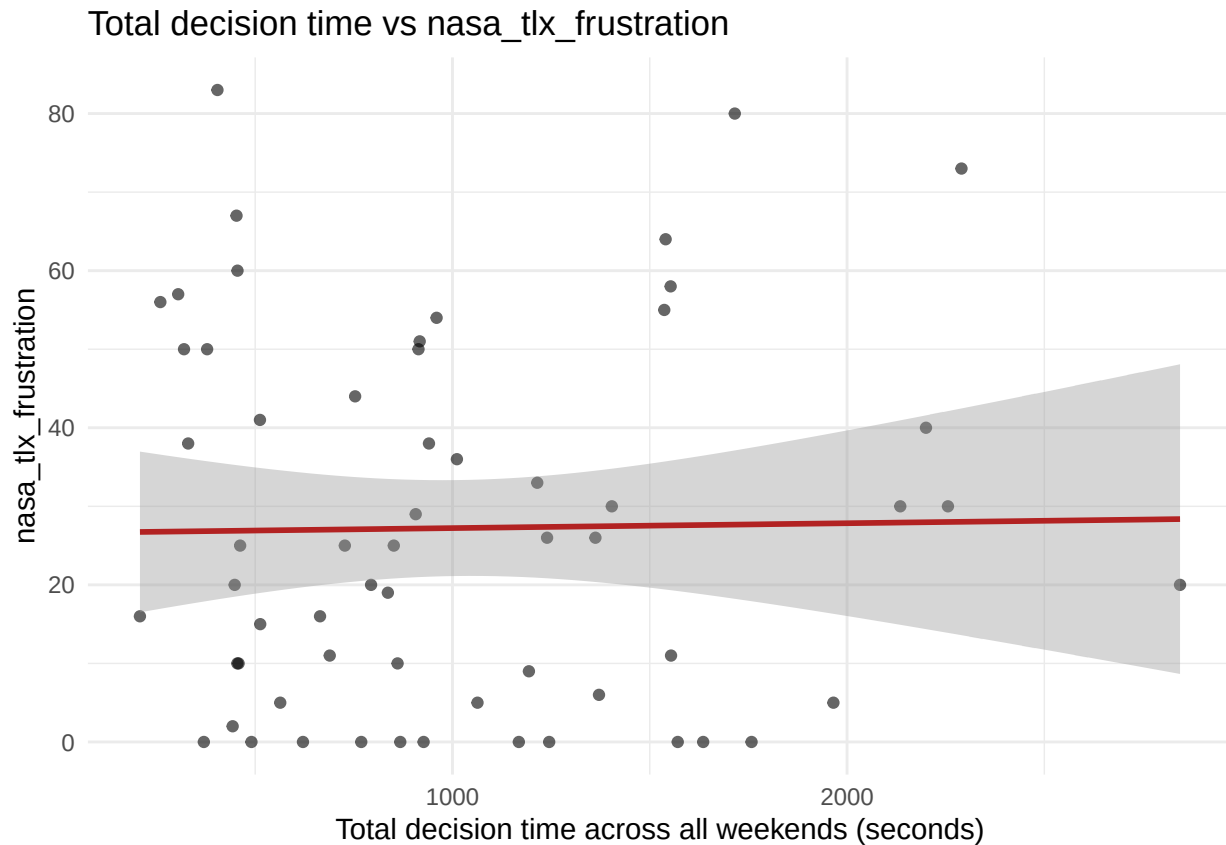


```
## `geom_smooth()` using formula = 'y ~ x'
```

Total decision time vs nasa_tlx_effort



```
## `geom_smooth()` using formula = 'y ~ x'
```



Testing H3.1 — Reduced-form analysis of the welfare effect of visual clutter in absence of cue

We test H3.1 using a fixed-effects logistic regression. The dependent variable is the binary indicator “Preference consistency”. The key regressor is “Visual Clutter” (Treatment 2). The unit of observation is subject $\times$ city, so each subject is observed 3 times. The model includes city-level fixed effects, which control for stable differences across cities that are unrelated to the treatment itself. We test whether preference consistency differs between clutter conditions using a two-sided test ($H_0 : \beta = 0$, $H_1 : \beta \neq 0$), with a Type I error rate of $\alpha = 0.10$ split across both tails.

```
# Subject  $\times$  city level: preference consistency is calculated at that level. Clutter condition is assigned
h31_df <- df %>%
  group_by(participant_code, city) %>%
  summarise(
    preference_consistency = first(preference_consistency),
    clutter_high = first(clutter_high),
    .groups = "drop"
  ) %>%
  mutate(participant_code = factor(participant_code), city = factor(city))

# NOTE: clutter_high is a BETWEEN-subject treatment - it never varies across a
# given subject's weekends/cities, so it is perfectly collinear with the
# participant_code fixed effect (each subject dummy already fully encodes their
# own clutter_high). Including participant_code here leaves zero residual
# variation to estimate beta, so we drop it and keep only the city fixed effect.

# NOTE: feglm(family = "logit") to match the spec ("fixed-effects logistic
# regression") - preference_consistency is binary, so OLS (feols) would be the
```

```

# wrong estimator here.
m_h31 <- feglm(
  preference_consistency ~ clutter_high | city,
  data = h31_df,
  family = "logit"
)

summary(m_h31)

## GLM estimation, family = binomial(link = "logit")
## Dep. Var.: preference_consistency
## Observations: 179
## Fixed-effects: city: 4
## Standard-errors: IID
##               Estimate Std. Error  z value Pr(>|z|)
## clutter_highTRUE 0.178035   0.307398 0.579167  0.56248
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood: -121.4   Adj. Pseudo R2: -0.030599
##               BIC: 268.8   Squared Cor.: 0.003055

# H3.1 is a two-sided test (H0: beta1 = 0, H1: beta1 != 0).
coef_h31 <- coef(m_h31)["clutter_highTRUE"]
se_h31 <- se(m_h31)["clutter_highTRUE"]
z_h31 <- coef_h31 / se_h31
p_twosided_h31 <- 2 * pnorm(abs(z_h31), lower.tail = FALSE)

cat("z =", round(z_h31, 3), " | two-sided p-value (H1: beta != 0) =", round(p_twosided_h31, 4), "\n")

## z = 0.579 | two-sided p-value (H1: beta != 0) = 0.5625

# Print mean preference consistency to audit
mean(h31_df$preference_consistency)

## [1] 0.4189944

```

Testing H3.2 — Reduced-form analysis of the welfare effect of cue visibility

We will test H3.2 using two one-sample binomial tests. We will test whether the frequency at which the binary indicator "Cued choice-preferences consistency" equals 1 to two fixed values. This frequency will be computed on the subject $\times$ city pair for with the "Preference consistency" indicator equals 1, no matter the Clutter condition. The fixed values are theoretically motivated. A value above the square root of the frequency of "Preference consistency" across all subject $\times$ city pair is interpreted as indicative of a positive welfare effect. A value below the square root of the frequency of "Preference consistency" divided by the square root of 3 is interpreted as indicative of a negative welfare effect. Intermediary values are interpreted as ambiguous.

```

# preference_consistency and cued_choice_preference_consistency are both
# constant within a (participant_code, city) pair, so collapse to 1 row per pair.
h32_df <- df %>%
  group_by(participant_code, city) %>%
  summarise(
    preference_consistency = first(preference_consistency),
    cued_choice_preference_consistency = first(cued_choice_preference_consistency),
    .groups = "drop"
  )

```



```

# Reference frequency: share of ALL subject x city pairs with a consistent
# revealed preference across the two no-cue weekends.
p_consistency <- mean(h32_df$preference_consistency, na.rm = TRUE)

# Restrict to pairs where preference_consistency == TRUE - cued_choice_preference_consistency
# is only defined (non-NA) there, since it's only meaningful when the subject has a
# well-defined preferred cluster to compare the cued choices against.
h32_eligible <- h32_df %>% filter(preference_consistency == TRUE)

n_eligible      <- nrow(h32_eligible)
n_cued_consistent <- sum(h32_eligible$cued_choice_preference_consistency, na.rm = TRUE)
freq_cued       <- n_cued_consistent / n_eligible

# Theoretically motivated thresholds from the pre-registration.
upper_threshold <- sqrt(p_consistency)
lower_threshold <- sqrt(p_consistency) / sqrt(3)

cat("p_consistency (all pairs):", round(p_consistency, 4), "\n")

## p_consistency (all pairs): 0.419
cat("Observed frequency (cued, among consistent pairs):", round(freq_cued, 4),
    "out of", n_eligible, "pairs\n")

## Observed frequency (cued, among consistent pairs): 0.4 out of 75 pairs
cat("Upper threshold (positive welfare):", round(upper_threshold, 4), "\n")

## Upper threshold (positive welfare): 0.6473
cat("Lower threshold (negative welfare):", round(lower_threshold, 4), "\n\n")

## Lower threshold (negative welfare): 0.3737

# Two one-sample binomial tests: is the observed frequency significantly
# different from each fixed, theoretically motivated value?
test_upper <- binom.test(n_cued_consistent, n_eligible, p = upper_threshold)
test_lower <- binom.test(n_cued_consistent, n_eligible, p = lower_threshold)

print(test_upper)

##
## Exact binomial test
##
## data: n_cued_consistent and n_eligible
## number of successes = 30, number of trials = 75, p-value = 1.581e-05
## alternative hypothesis: true probability of success is not equal to 0.6472978
## 95 percent confidence interval:
## 0.2885192 0.5196295
## sample estimates:
## probability of success
## 0.4
print(test_lower)

##
## Exact binomial test

```

```
##
## data:  n_cued_consistent and n_eligible
## number of successes = 30, number of trials = 75, p-value = 0.6349
## alternative hypothesis: true probability of success is not equal to 0.3737175
## 95 percent confidence interval:
##  0.2885192 0.5196295
## sample estimates:
## probability of success
##                0.4

# Interpretation per the pre-registration rule.
welfare_conclusion <- dplyr::case_when(
  freq_cued > upper_threshold ~ "Positive welfare effect",
  freq_cued < lower_threshold ~ "Negative welfare effect",
  TRUE ~ "Ambiguous"
)

cat("\nConclusion:", welfare_conclusion, "\n")

##
## Conclusion: Ambiguous
```

Manipulation/ Internal validity checks

Clutter perception: To verify that Treatment 2 successfully manipulates perceived visual complexity, we compare subjects in the Cluttered and Low Clutter groups on their answers to the post-experiment questionnaire item “How visually complex did you find the website interface?” (7-point Likert scale). We test whether the mean difference between groups is non-zero using a two-sided two-sample t-test, with a Type I error rate of $= 0.10$.

```
# visual_complexity is collected once per participant (post-experiment
# questionnaire) -> collapse to 1 row per subject.
h_manip_df <- df %>%
  group_by(participant_code) %>%
  summarise(
    visual_complexity = first(visual_complexity),
    clutter_high = first(clutter_high),
    .groups = "drop"
  )

# Two-sided, two-sample t-test (Welch by default) comparing perceived visual
# complexity between High Clutter and Low Clutter subjects.
t_manip <- t.test(visual_complexity ~ clutter_high, data = h_manip_df)
print(t_manip)

##
## Welch Two Sample t-test
##
## data:  visual_complexity by clutter_high
## t = 0.6542, df = 57.84, p-value = 0.5156
## alternative hypothesis: true difference in means between group FALSE and group TRUE is not equal to 0
## 95 percent confidence interval:
##  -0.6530562  1.2870940
## sample estimates:
## mean in group FALSE  mean in group TRUE
##          2.903226          2.586207
```

```
cat("Mean visual complexity - High Clutter:",
    round(mean(h_manip_df$visual_complexity[h_manip_df$clutter_high], na.rm = TRUE), 2),
    "| Low Clutter:",
    round(mean(h_manip_df$visual_complexity[!h_manip_df$clutter_high], na.rm = TRUE), 2), "\n")
```

```
## Mean visual complexity - High Clutter: 2.59 | Low Clutter: 2.9
```

Preference consistency: As validation that the clusters capture meaningful and stable consumer preferences, we report the average preference consistency rate across all subject $\times$ city pairs — that is, the share of subject $\times$ city pairs for which the two choices made in no-cue weekends belong to the same cluster. A high consistency rate provides evidence that clusters reflect genuine preference types and that subjects make coherent choices across weekends in the absence of the cue. We test that preference consistency is higher than chance (corresponding to a random allocation of listings to clusters) using a one-sample, one-sided t-test comparing the mean preference consistency with $\frac{1}{3}$, with a Type I error rate of $\alpha = 0.10$.

```
# preference_consistency is constant within a (participant_code, city) pair,
# so collapse to 1 row per pair before averaging (otherwise the 108 listing-rows
# per pair would just repeat the same value and not bias the mean, but it's
# cleaner/cheaper to collapse first).
h_consistency_df <- df %>%
  group_by(participant_code, city) %>%
  summarise(preference_consistency = first(preference_consistency), .groups = "drop")

consistency_rate <- mean(h_consistency_df$preference_consistency, na.rm = TRUE)
n_pairs <- sum(!is.na(h_consistency_df$preference_consistency))

cat("Preference consistency rate:", round(consistency_rate * 100, 1), "%",
    "(", n_pairs, "subject x city pairs )\n")
```

```
## Preference consistency rate: 41.9 % ( 179 subject x city pairs )
```

```
# One-sample, one-sided t-test: is preference consistency higher than chance
# (1/3, the rate expected under random allocation of listings to clusters)?
t_consistency <- t.test(
  h_consistency_df$preference_consistency,
  mu = 1/3,
  alternative = "greater"
)
print(t_consistency)
```

```
##
## One Sample t-test
##
## data: h_consistency_df$preference_consistency
## t = 2.3163, df = 178, p-value = 0.01084
## alternative hypothesis: true mean is greater than 0.3333333
## 95 percent confidence interval:
## 0.3578471 Inf
## sample estimates:
## mean of x
## 0.4189944
```